

Big Data Analytics – Mini Project 1

The Digital Librarian (Distributed Reverse Indexing)

Student Name: khaled mohsen
omar eldesouky

Course: Big Data Analytics

Date: March 2026

1. Introduction / Problem Definition

Traditional text search methods such as sequential scanning (e.g., using grep) are inefficient when dealing with large-scale datasets. As the size of digital libraries grows, the time required to search for keywords increases significantly, making such approaches impractical.

To address this problem, a **Reverse Index** is used. Unlike a forward index, which maps documents to words, a reverse index maps each word to the list of documents it appears in along with its frequency.

This structure is widely used in search engines, enabling fast and scalable keyword-based retrieval for users such as researchers, librarians, and general users.

2. Data Science Lifecycle Implementation

2.1 Storage Analysis (HDFS)

The dataset consists of multiple text files (books). These files are stored in HDFS, which provides:

- **Block Storage:** Files are split into blocks for parallel processing
- **Replication:** Ensures fault tolerance
- **Data Locality:** Computation is moved closer to data

This makes HDFS suitable for large-scale text processing.

2.2 Data Preparation (Preprocessing)

The following preprocessing steps were applied in the Mapper:

1. **Normalization**
 - a. Converted text to lowercase
 - b. Removed punctuation
2. **Stop-word Removal**
 - a. Common words such as "the", "is", "at" were removed
 - b. A file stopwords.txt was used and loaded into memory

Impact:

Stop-word removal significantly reduced the number of intermediate key-value pairs, which:

- Reduced shuffle size
- Decreased network overhead
- Improved efficiency

2.3 Processing Logic (MapReduce)

Mapper:

- Reads text line by line
- Tokenizes words
- Emits:

(word, document_name)

Shuffle & Sort:

- Groups all values by key (word)

Reducer:

- Counts occurrences of each word in each document
- Outputs:

word → document1:count, document2:count

3. Scalability Analysis & Results

3.1 Methodology

Due to environment limitations, horizontal scaling was simulated by increasing dataset size:

Run	Dataset
Run 1	Small (input1.txt)
Run 2	Medium (input2.txt)
Run 3	Large (input3.txt)

Execution time was measured using the `time` command.

3.2 Results

Run	Execution Time (seconds)
T1	15.99
T2	17.25
T3	23.71

3.3 Speedup Calculation

$$S_2 = T_1 / T_2 = 15.99 / 17.25 \approx 0.93$$

$$S_3 = T_1 / T_3 = 15.99 / 23.71 \approx 0.67$$

3.4 Analysis

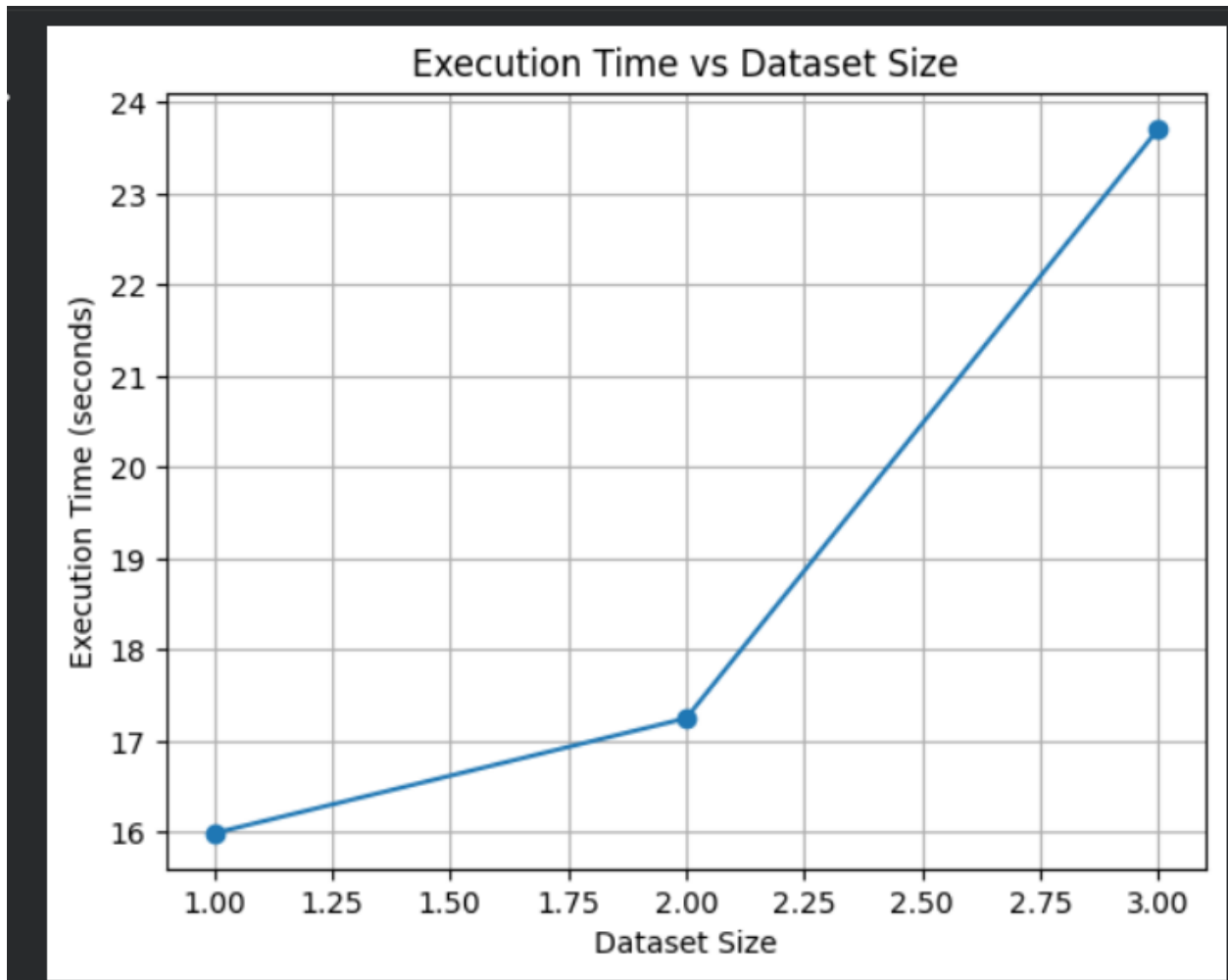
- Execution time increased with input size
- No linear speedup was achieved
- Performance degraded due to:
 - Single-node execution (no real distribution)
 - Shuffle overhead
 - Disk I/O
 - Task initialization overhead

3.5 Amdahl's Law

The results align with **Amdahl's Law**, which states that the sequential portion of a program limits overall speedup.

Since the system was not truly distributed, the sequential overhead dominated performance.

3.6 Graph Description



- X-axis: Dataset Size (1, 2, 3)
- Y-axis: Execution Time (16, 17, 24)

The graph shows an increasing trend, indicating lack of scalability.

4. Conclusion

This project demonstrated the use of Hadoop and MapReduce to build a Reverse Index for large-scale text processing.

Key findings:

- HDFS enables scalable storage
- MapReduce simplifies distributed processing
- Stop-word removal improves efficiency
- Lack of true distribution limits performance

Overall, distributed indexing is effective, but scalability depends on proper cluster configuration.

5. Files Included

- mapper.py
- reducer.py
- stopwords.txt
- input datasets
- output results